04 주차 교재 3.3–3.6절 (p.90~)

가우시안 필터 II — EKF · UKF · 정보 필터

선형이 아닌 세계에 칼만 필터를 밀어 넣는 세 가지 방법

이번 주 학습목표

1. 선형화가 왜 필요한지 설명하고, 테일러 전개에 기반한 EKF의 야코비안 G_t, H_t를 직접 유도한다.
2. 선형화 오차가 사후분포를 어떻게 왜곡하는지 정성적으로 설명하고, 오차 크기를 좌우하는 두 요인을 지목한다.
3. 분산점(sigma point) 변환의 원리를 설명하고, UKF가 EKF보다 정확한 이유를 모멘트 관점에서 서술한다.
4. 모멘트 파라미터화와 캐노니컬 파라미터화의 쌍대성을 이해하고, 정보 필터에서 측정 갱신이 덧셈이 되는 이유를 유도한다.
5. 네 가지 필터(KF·EKF·UKF·EIF)의 계산 복잡도와 적용 조건을 표로 정리하고 상황에 맞게 선택한다.

> **자율주행 연결**
>
> 차량 운동학은 선형이 아니다. 조향각이 들어간 자전거 모델은 삼각함수를 포함하므로 3.2절의 첫 번째 가정이 곧바로 깨진다. ROS 2의 `robot_localization` 패키지가 EKF와 UKF 두 노드를 함께 제공하는 이유가 여기에 있으며, 실무에서 둘 중 무엇을 고를지는 4.5절의 비교표로 판단한다. **정보 필터는 13주차 GraphSLAM의 언어가 되므로** 지금 캐노니컬 파라미터화에 익숙해질 것.

4.1 선형화가 필요한 이유

비선형 함수를 통과한 가우시안은 **더 이상 가우시안이 아니다**. 이것이 문제의 전부다.

이제 시스템을 일반적인 비선형 형태로 쓴다.

$$x_t = g(u_t,\ x_{t-1}) + \varepsilon_t, \qquad z_t = h(x_t) + \delta_t \tag{4.1}$$

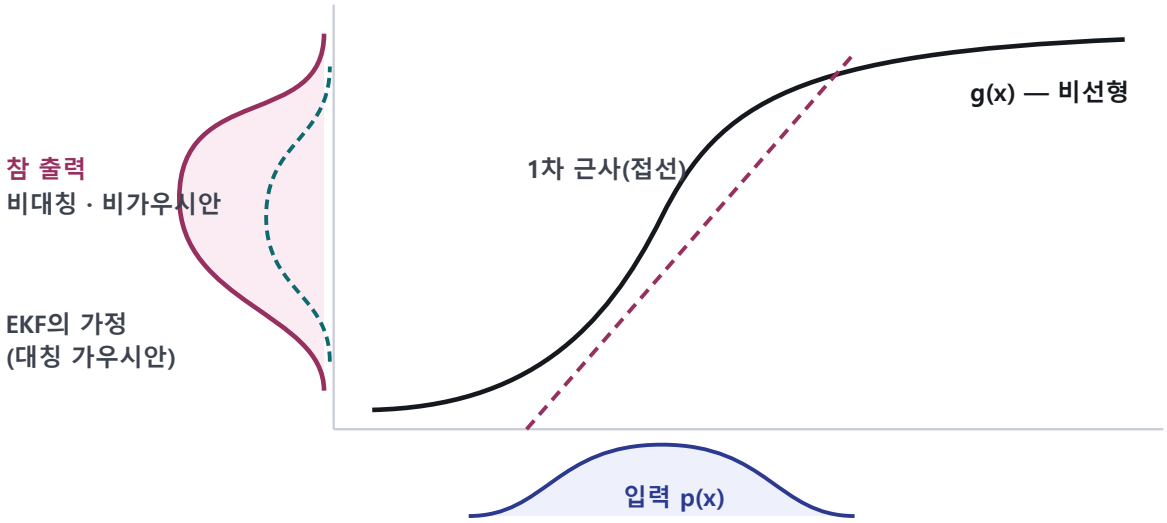


그림 4.1 비선형 변환 후의 참 분포(자홍)는 비대칭이지만, EKF는 접선 근사를 통해 이를 대칭 가우시안(청록)으로 강제한다. 두 분포의 차이가 곧 **선형화 오차**이며, 이 오차는 필터가 스스로 인식하지 못한 채 누적된다.

선형화 오차를 키우는 두 요인

1. **함수의 국소 비선형성** — 곡률이 클수록 접선 근사가 나빠진다.
2. **사전분포의 불확실성** $\bar{\Sigma}$ — 분포가 넓게 퍼져 있을수록 접선이 유효한 범위를 벗어난 영역까지 근사하게 된다.

둘째 요인이 실무적으로 더 중요하다. **불확실성이 큰 상태에서 EKF를 시작하면 발산하기 쉽다**는 뜻이며, 이것이 EKF가 전역 로컬화에 부적합한 두 번째 이유다(첫 번째는 단봉성).

4.2 확장형 칼만 필터 (EKF)

테일러 전개

모션 함수를 사전 평균 μ_{t-1} 주위에서 1차까지 전개한다.

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + \underbrace{G_t}_{\text{야코비안}}(x_{t-1} - \mu_{t-1}), \qquad G_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}} \tag{4.2}$$

측정 함수도 예측 평균 $\bar{\mu}_t$ 주위에서 전개한다.

$$h(x_t) \approx h(\bar{\mu}_t) + H_t(x_t - \bar{\mu}_t), \qquad H_t = \frac{\partial h(\bar{\mu}_t)}{\partial x_t} \tag{4.3}$$

> **전개 기준점을 놓치지 말 것**
>
> G_t는 μ_{t-1}에서, H_t는 $\bar{\mu}_t$에서 평가한다. 서로 다른 시점의 서로 다른 추정값이다. 구현에서 이 둘을 혼동하면 필터가 미묘하게 편향되며, 잡음이 큰 상황에서만 드러나기 때문에 디버깅이 매우 어렵다.

Algorithm Extended_Kalman_filter(μ_{t-1}, Σ_{t-1}, u_t, z_t):

1. $\bar{\mu}_t = g(u_t, \mu_{t-1})$ ← 비선형 함수를 **그대로** 사용
2. $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t$ ← 공분산 전파에만 야코비안 사용
3. $K_t = \bar{\Sigma}_t H_t^\top \left(H_t \bar{\Sigma}_t H_t^\top + Q_t \right)^{-1}$
4. $\mu_t = \bar{\mu}_t + K_t \left(z_t - h(\bar{\mu}_t) \right)$ ← 혁신도 비선형 함수로
5. $\Sigma_t = (I - K_t H_t)\, \bar{\Sigma}_t$
6. return μ_t, Σ_t

칼만 필터와 비교하면 대응이 명확하다: $A_t \to G_t$, $C_t \to H_t$, 그리고 **평균 계산에는 야코비안 대신 원래 함수를 쓴다**. 이 마지막 점이 EKF의 실용적 정확도를 크게 높인다.

예제 — 이동로봇의 야코비안

속도 모션 모델(원호 운동)을 쓰는 이동로봇을 보자. $x = (x, y, \theta)^\top$, 제어 $u = (v, \omega)^\top$일 때

$$g(u, x) = \begin{pmatrix} x - \frac{v}{\omega}\sin\theta + \frac{v}{\omega}\sin(\theta + \omega\Delta t) \\ y + \frac{v}{\omega}\cos\theta - \frac{v}{\omega}\cos(\theta + \omega\Delta t) \\ \theta + \omega\Delta t \end{pmatrix}$$

이를 x에 대해 미분하면

$$G_t = \begin{pmatrix} 1 & 0 & -\frac{v}{\omega}\cos\theta + \frac{v}{\omega}\cos(\theta+\omega\Delta t) \\ 0 & 1 & -\frac{v}{\omega}\sin\theta + \frac{v}{\omega}\sin(\theta+\omega\Delta t) \\ 0 & 0 & 1 \end{pmatrix}$$

구현 시 함정

- $\omega \to 0$에서 v/ω가 발산한다. $|\omega| < 10^{-6}$이면 직진 모델로 분기하거나 극한값을 쓸 것.
- **각도 정규화** — 혁신 $z_t - h(\bar{\mu}_t)$에 각도 성분이 있으면 반드시 $(-\pi, \pi]$로 감싸야 한다. 이를 빠뜨리면 179°와 -179°의 차이가 358°로 계산되어 필터가 즉시 발산한다. **EKF 로컬화 과제에서 가장 흔한 버그**다.
- **공분산의 대칭성** — 수치 오차로 Σ가 비대칭이 되면 서서히 망가진다. 매 스텝 `Sig = 0.5*(Sig + Sig.T)` 로 강제할 것.

4.3 무향 칼만 필터 (UKF)

EKF는 함수를 근사한다. UKF는 발상을 뒤집어 **분포를 근사**한다.

> **핵심 아이디어**
>
> 가우시안을 대표하는 소수의 결정론적 점(**분산점, sigma point**)을 고른 뒤, 그 점들을 **비선형 함수에 그대로 통과**시키고, 나온 점들로부터 평균과 공분산을 다시 계산한다. 야코비안이 필요 없다.

n차원 가우시안 (μ, Σ)에 대해 $2n+1$개의 분산점을 잡는다.

$$\mathcal{X}^{[0]} = \mu, \qquad \mathcal{X}^{[i]} = \mu + \left(\sqrt{(n+\lambda)\Sigma}\right)_i, \qquad \mathcal{X}^{[i+n]} = \mu - \left(\sqrt{(n+\lambda)\Sigma}\right)_i \tag{4.4}$$

여기서 $i = 1, \dots, n$이고 $\left(\sqrt{\cdot}\right)_i$는 행렬 제곱근(보통 Cholesky 분해)의 i번째 열이다.
$\lambda = \alpha^2(n+\kappa) - n$은 분산점의 퍼짐 정도를 정하는 척도 파라미터다.

각 점에 평균용·공분산용 가중치를 부여한다.

$$w_m^{[0]} = \frac{\lambda}{n+\lambda}, \qquad w_c^{[0]} = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta), \qquad w_m^{[i]} = w_c^{[i]} = \frac{1}{2(n+\lambda)}$$

변환 후 모멘트는 가중합으로 복원한다.

$$\mu' = \sum_{i=0}^{2n} w_m^{[i]}\, g(\mathcal{X}^{[i]}), \qquad \Sigma' = \sum_{i=0}^{2n} w_c^{[i]} \left(g(\mathcal{X}^{[i]}) - \mu'\right)\left(g(\mathcal{X}^{[i]}) - \mu'\right)^{\mathsf{T}} \tag{4.5}$$

파라미터	전형값	역할
α	$10^{-3} \sim 1$	분산점이 평균에서 얼마나 멀리 퍼지는가. 작을수록 국소적.
κ	0 또는 $3-n$	보조 척도 파라미터. 고차 모멘트 정합에 관여.
β	2	사전분포의 형태에 대한 지식. 가우시안이면 2가 최적.

> **EKF VS UKF — 무엇이 다른가**
>
> - **정확도** — UKF는 테일러 전개 기준 **3차 항까지** 평균을 정확히 맞춘다(가우시안 입력일 때). EKF는 1차까지만이다.
> - **야코비안 불필요** — 미분 불가능하거나 해석적 미분이 번거로운 함수(예: 룩업 테이블, 시뮬레이터)에도 적용 가능하다. 실무에서 이 장점이 정확도보다 더 자주 채택 이유가 된다.

- **계산량** — 비선형 함수를 $2n+1$회 평가해야 하므로 EKF보다 무겁다. 다만 야코비안 계산 비용을 아끼므로 실제 차이는 크지 않다.
- **공통 한계** — UKF도 결국 결과를 **가우시안으로 요약**한다. 다봉 분포는 여전히 표현하지 못한다.

4.4 정보 필터 — 쌍대적 표현

가우시안을 평균·공분산이 아니라 다음 두 양으로 표현할 수 있다.

$$\Omega = \Sigma^{-1} \quad (\text{정보행렬}), \qquad \xi = \Sigma^{-1}\mu \quad (\text{정보벡터}) \tag{4.6}$$

이를 **캐노니컬 파라미터화**라 한다. 밀도함수는

$$p(x) = \text{const} \cdot \exp\left(-\tfrac{1}{2}x^{\mathsf{T}}\Omega x + x^{\mathsf{T}}\xi\right)$$

왜 이 형태가 유용한가

3주차 식 (3.5)를 다시 보자.

$$\Sigma_t^{-1} = \bar{\Sigma}_t^{-1} + C_t^{\mathsf{T}} Q_t^{-1} C_t \quad \Longleftrightarrow \quad \Omega_t = \bar{\Omega}_t + C_t^{\mathsf{T}} Q_t^{-1} C_t \tag{4.7}$$

> **측정 갱신이 덧셈이다**
>
> 정보 폼에서 측정을 반영하는 일은 **정보행렬에 항을 더하는 것**에 불과하다. 따라서:
>
> - **다중 센서 융합이 자명하다** — 서로 독립인 K개의 측정은 K개의 항을 그냥 더하면 된다. 순서도 상관없다.
> - **측정 차원이 매우 커도 싸다** — 역행렬 크기가 측정 차원에 의존하지 않는다.
> - **"정보가 없음"을 표현할 수 있다** — $\Omega = 0$은 완전 무지(무한 공분산)를 뜻한다. 모멘트 폼에서는 $\Sigma = \infty$를 다룰 수 없다.

Algorithm Extended_information_filter(ξ_{t-1}, Ω_{t-1}, u_t, z_t):

1. $\mu_{t-1} = \Omega_{t-1}^{-1}\xi_{t-1}$ ← 비선형 함수 평가를 위해 **복원 필요**
2. $\bar{\Omega}_t = \left(G_t \Omega_{t-1}^{-1} G_t^{\mathsf{T}} + R_t\right)^{-1}$ ← 역행렬 두 번, 비싸다
3. $\bar{\xi}_t = \bar{\Omega}_t \, g(u_t, \mu_{t-1})$
4. $\Omega_t = \bar{\Omega}_t + H_t^{\mathsf{T}} Q_t^{-1} H_t$ ← 덧셈
5. $\xi_t = \bar{\xi}_t + H_t^{\mathsf{T}} Q_t^{-1}\left[z_t - h(\bar{\mu}_t) + H_t \bar{\mu}_t\right]$
6. return ξ_t, Ω_t

> **대가 — 예측 단계가 비싸다**
>
> 1–2행에서 $n \times n$ 역행렬이 두 번 등장한다. EKF에서는 예측이 싸고 보정이 비쌌는데, 정보 필터는 정확히 반대다. **어느 쪽도 공짜가 아니다.**

> 그렇다면 왜 배우는가? 14주차 SEIF 때문이다. SLAM에서 정보행렬은 **거의 대각에 가까운 희소 구조**를 가지며(로봇은 근처 랜드마크만 관측하므로), 이 희소성을 유지하면 예측 단계의 비용까지 $O(1)$로 만들 수 있다. 그것이 SEIF의 아이디어다.

4.5 네 필터 비교

	KF	EKF	UKF	EIF
표현	μ, Σ	μ, Σ	μ, Σ	ξ, Ω
비선형 처리	불가	1차 테일러	분산점 (3차)	1차 테일러
야코비안	불필요	**필요**	불필요	**필요**
예측 비용	싸다	싸다	중간	**비싸다**
보정 비용	$O(k^{2.4})$	$O(k^{2.4})$	$O(k^{2.4})$	**싸다(덧셈)**
다봉 표현	모두 **불가능** — 5주차 비모수 필터가 필요한 이유			
이 강의에서	3주	9주 로컬화 12주 EKF SLAM	9주 UKF 로컬화	14주 SEIF

실습 — 비선형 시스템에서 EKF와 UKF 비교

```
# ekf_vs_ukf.py — 극좌표 관측을 갖는 2D 추적
import numpy as np

# 스테이트 x = (px, py, vx, vy),  관측 z = (range, bearing)
def h(x):
    r  = np.hypot(x[0], x[1])
    b  = np.arctan2(x[1], x[0])
    return np.array([r, b])

def H_jac(x):
    px, py = x[0], x[1]
    r2 = px**2 + py**2; r = np.sqrt(r2)
    return np.array([[ px/r,   py/r,  0, 0],
                     [-py/r2,  px/r2, 0, 0]])

def wrap(a):
    '''각도를 (-pi, pi]로 — 빠뜨리면 필터가 발산한다'''
    return (a + np.pi) % (2*np.pi) - np.pi

def sigma_points(mu, Sig, alpha=1e-3, kappa=0.0, beta=2.0):
    n = len(mu)
    lam = alpha**2 * (n + kappa) - n
    S = np.linalg.cholesky((n + lam) * Sig)
    X = np.vstack([mu, mu + S.T, mu - S.T])          # (2n+1, n)
    wm = np.full(2*n+1, 1/(2*(n+lam))); wm[0] = lam/(n+lam)
    wc = wm.copy();  wc[0] = lam/(n+lam) + (1 - alpha**2 + beta)
```

```
    return X, wm, wc

def ukf_correct(mu_b, Sig_b, z, Q):
    X, wm, wc = sigma_points(mu_b, Sig_b)
    Z = np.array([h(x) for x in X])
    z_hat = wm @ Z
    dZ = Z - z_hat;  dZ[:, 1] = wrap(dZ[:, 1])
    dX = X - mu_b
    S   = (wc[:,None,None] * np.einsum('ki,kj→kij', dZ, dZ)).sum(0) + Q
    Cxz = (wc[:,None,None] * np.einsum('ki,kj→kij', dX, dZ)).sum(0)
    K = Cxz @ np.linalg.inv(S)
    innov = z - z_hat;  innov[1] = wrap(innov[1])
    return mu_b + K @ innov, Sig_b - K @ S @ K.T
```

> **실험 설계**
>
> 로봇이 원점 근처를 지날 때 방위각 $b = \arctan 2(p_y, p_x)$의 비선형성이 극단적으로 커진다. 이 구간에서 EKF와 UKF의 **NEES(정규화 추정 오차 제곱)**를 비교하면 EKF의 과신(overconfidence) 현상 — 실제 오차보다 공분산이 작게 나오는 — 을 정량적으로 확인할 수 있다.

연습문제 (과제 1 · 4주차 제출)

1. 기초 위 코드를 완성해 EKF와 UKF를 같은 데이터에 돌리고 RMSE를 비교하라.
2. 기초 `wrap` 함수를 제거하면 어떤 일이 일어나는가? 직접 확인하고 그 이유를 서술하라.
3. 응용 초기 공분산 Σ_0를 10배, 100배 키우면서 두 필터의 발산 여부를 관찰하라. 4.1절에서 말한 "두 번째 요인"이 확인되는가?
4. 응용 UKF의 α를 10^{-3}에서 1까지 바꾸며 성능 변화를 측정하라. Cholesky 분해가 실패하는 경우가 있는가?
5. 심화 두 필터의 NEES를 계산하고 χ^2 신뢰구간과 비교해 **일관성(consistency)**을 검정하라. 어느 필터가 자신의 오차를 더 정직하게 보고하는가?
6. 심화 (4.7)의 정보 폼 측정 갱신을 유도하고, 서로 독립인 두 측정을 **순서를 바꿔** 반영해도 결과가 같음을 증명하라. EKF에서는 왜 이것이 자명하지 않은가?

4.6 정리와 다음 주 연결

> **한 문장 요약**
>
> EKF는 함수를 선형화하고, UKF는 분포를 샘플링하며, 정보 필터는 표현을 뒤집는다. 셋 다 유용하지만 **단봉 가우시안이라는 근본 제약은 공유한다**.

5주차에서는 이 제약을 정면으로 걷어낸다. 스테이트 공간을 이산화하는 **히스토그램 필터**와, 샘플 집합으로 임의 분포를 표현하는 **입자 필터**를 다룬다. 입자 필터는 10주차 MCL과 15주차 FastSLAM의 기반이 되므로, 이번 학기에서 두 번째로 중요한 도구다.

교재 4.1–4.4절을 사전 통독하고, 특히 **중요도 샘플링(importance sampling)**의 논리 — 원하는 분포에서 직접 뽑을 수 없을 때 다른 분포에서 뽑고 가중치로 보정한다 — 를 미리 이해해 올 것.

공주대학교 대학원 · 자율주행 · 담당 이세진

주교재: S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005. (한국어판 『확률론적 로보틱스』)